合肥师范学院·正常模式 · 答案与解析

一、单项选择题（15×2=30分）

1.

B = -74（取反加一漏 "+1"）

错误路径：数值位取反得 1001010（=74），但漏掉最后一步 "+1"，直接取负：

-74

正确值应为 -75，因漏 "+1" 导致绝对值少 1，得 -74。

C = -53（补码当作原码直接读）

错误路径：将补码机器数当作原码处理——符号位 1 表负，数值位 0110101 直接读为 53：

-53

正确值 -75 ≠ -53，混淆了补码与原码的数值位解释。

D = 181（符号位权重取 $+2^7$ 而非 -2^7）

错误路径：加权求和时所有位（含符号位）均取正权重：

$1\times2^7 + 0\times2^6 + 1\times2^5 + 1\times2^4 + 0\times2^3 + 1\times2^2 + 0\times2^1 + 1\times2^0$

= 128 + 32 + 16 + 4 + 1 = 181

符号位本应贡献 -128，误取 +128，导致结果为正且数值错误。

A. -75

2.

参数	值
操作数 A	`0x41266666` → float = 10.399999618530273
操作数 B	`0x3F8CCCCC` → float = 1.0999999046325684
A + B（Python 精确参考）	11.499999523162842
A + B（Python float hex）	`0x41380000`

分解各操作数（exec OPTION A 数据）

字段	操作数 A	操作数 B
符号位	0	0

字段	操作数 A	操作数 B
阶码字段（8 位）	130	127
真值指数（阶码 − 127）	3	0
24 位完整尾数（含隐含 1）	`101001100110011001100110`	`100011001100110011001100`

· 两操作数阶码差 ediff = true_exp_a − true_exp_b = 3 − 0 = **3**，满足 ∈ [2, 5]。

· 对阶方向：小阶向大阶看齐 → B 的尾数右移 3 位。

选项	十六进制值	描述	判定
A	**`0x41380000`**	正确结果	✅ **正确**
B	`0x3FB80000`	对阶方向错误	❌
C	`0x40D00000`	忽略隐含位	❌
D	`0x4137FFFF`	不执行舍入（truncation）	❌

答案：A（`0x41380000`）

3.

答案：C **解析**：双符号位（变形补码）的溢出判定规则为：两个符号位相同（00 或 11）表示未溢出，两个符号位不同（01 或 10）表示溢出。其中 01 表示正溢出（上溢），10 表示负溢出（下溢）。题干中符号位为 10，即发生了负溢出（下溢），C 正确。A 错误，将 10 误判为未溢出；B 错误，将 10 的溢出方向误判为正溢出（01 才是正溢出）；D 错误，10 属于溢出状态，并非正常结果。

4.

选项 B：地址线 / 数据线互换（概念混淆）

将正确结果中的地址线数与数据线数对调：

· 地址线 = 32（取了正确的数据线数）

· 数据线 = 15（取了正确的地址线数）

选项 B：地址线 32 根，数据线 15 根。 错误原因：混淆了地址线和数据线的概念。

选项 C：K = 1000 单位换算错误

将 1K 误当作 1000，则总字节数 = 32 × 1000 = 32000。

$\log_2(32000) \approx 14.97$

取整数部分得地址线数 = 14，数据线数不变 = 32。

选项 C：地址线 14 根，数据线 32 根。 错误原因：单位换算中将 1K = 1024 误作 1000，导致编址单元总数偏小，地址线数算少 1 根。

选项 D：按字编址理解偏差

误认为按字编址，编址单元 = 1 个存储字 = 32 位 = 4 字节。

编址单元总数 = 32768 / 4 = 8192

地址线数 = $\log_2(8192) = 13$

数据线数不变 = 32。

选项 D：地址线 13 根，数据线 32 根。 错误原因：编址方式理解偏差，将按字节编址误作按字编址，编址单元粒度变大导致编址单元数减少，地址线数算少 2 根。

A. 地址线 15 根，数据线 32 根。

选项	地址线	数据线	正误
A	**15**	**32**	**✓ 正确**
B	32	15	✗ 互换
C	14	32	✗ K=1000
D	13	32	✗ 按字编址

5.

答案：C **解析**：SRAM 利用触发器存储信息，只要不断电信息就不会丢失，不需要刷新；DRAM 利用电容存储信息，电容存在漏电现象，必须定期刷新以维持数据，C 正确。A 错误，将 SRAM 和 DRAM 的存储原理完全说反了；B 错误，DRAM 的集成度高于 SRAM（一个 DRAM 存储单元只需一个晶体管和一个电容），且 DRAM 速度通常慢于 SRAM；D 错误，SRAM 不需要刷新。

6.

答案：B **解析**：直接映射方式下，每个主存块只能映射到固定的某一行（由主存块号 mod Cache 行数决定），但能否命中取决于该 Cache 行当前存放的内容是否为该主存块，即标记（Tag）是否匹配，B 正确。A 错误，块号与行号相同只是映射关系，不代表该行当前存的就是这个块；C 错误，命中率取决于访存模式与替换情况，不是固定值；D 错误，描述的是全相联映射的特征，不是直接映射。

7.

第 1 级（累积宽度 3 位）

- 可用编码总数 = $2^3 = 8$
- 需预留 1 个编码用于向下扩展（第 1 级是非末级）

· L1 = 8 − 1 = **7 条**

第 2 级（累积宽度 5 位）— 目标级别

· 新增位数 = o2 − o1 = 5 − 3 = **2 位**

· 入口预留前缀数 = 第 1 级预留的 **1 个**编码

· 编码空间 = $1 \times 2^2 = 4$

· 需预留 1 个编码用于向第 3 级扩展（第 2 级是非末级）

· L2 = 4 − 1 = **3 条**

> **关键规则**：第 2 级是非末级，必须扣留 ⩾1 个编码作为向第 3 级扩展的前缀，所以 $2^2 = 4$ 个编码中只能用 3 个定义指令。

第 3 级（累积宽度 8 位，末级）

· 新增位数 = o3 − o2 = 8 − 5 = 3 位

· 入口预留前缀数 = 第 2 级预留的 1 个编码

· 编码空间 = $1 \times 2^3 = 8$

· 末级无需预留

· L3 = **8 条**（参考，非本题所问）

8.

答案：C **解析**：变址寻址的有效地址 EA = (RX) + D = 0010H + 0300H = 0310H，C 正确。A 错误，仅取形式地址，遗漏了变址寄存器的内容（对应直接寻址）；B 错误，仅取变址寄存器内容，遗漏了形式地址（对应寄存器间接寻址的效果）；D 错误，将加法误算为减法，方向错误。

9.

答案：B **解析**：PC（程序计数器）用于存放下一条要取的指令的地址（在指令取出后通常自动递增），IR（指令寄存器）用于存放当前从主存中取出的指令，B 正确。A 错误，将 PC 和 IR 的功能完全颠倒；C 错误，PC 存地址、IR 存指令，二者内容和功能不同；D 错误，PC 不存放操作码，IR 存放的是完整指令（含操作码和地址码）。

10.

第一步：识别关键量

· 各段延迟之和：$\Sigma\Delta t_i = 3+2+5+2+3 = 15$ ns

· 瓶颈段延迟：$\Delta t_{max} = \max(\Delta t_i) = 5$ ns（执行段 EX）

· 平均延迟：$avg = \Sigma\Delta t_i / k = 15 / 5 = 3.0$ ns

第二步：计算流水线总执行时间

各段不等时流水线，总执行时间公式：

$T = \Sigma\Delta t_i + (N-1) \times \Delta t_{max}$

$T = 15 + (50-1) \times 5 = 15 + 49 \times 5 = 15 + 245 = 260$ ns

第三步：计算吞吐率

$TP = N/T = 50/260 = 0.192308$ 条/ns

换算为 $\times 10^8$ 条/秒（1 ns = 10^{-9} s，故 1 条/ns = 10^9 条/s = 10 $\times 10^8$ 条/s）：

$TP = 0.192308 \times 10 = 1.9231 \times 10^8$ 条/s

A. 1.92 $\times 10^8$ 条/s

选项	数值	计算路径	正误
A	**1.92**	TP = N/T = 50/260 = 0.1923 条/ns = **1.923 $\times 10^8$** 条/s	✅ 正确
B	28.85	加速比 $S = N\Sigma\Delta t_i / T = 50 \times 15/260 = 2.885$，误将 S 数值当 TP：$2.885 \times 10 = 28.85 \times 10^8$	❌ 加速比与吞吐率混淆
C	3.09	取平均延迟 avg=3.0 替代最大值：T'=15+49×3.0=162 ns，TP'=50/162=0.3086 条/ns = 3.086×10^8	❌ 瓶颈段取错
D	2.00	遗漏首条指令填充时间，仅用 $N \times \Delta t_{max}$=250 ns 作为总时间：TP''=50/250=0.2 条/ns = 2.0×10^8	❌ 总时间遗漏填充

错误路径详细说明

- **B 错误**：加速比 $S = N\Sigma\Delta t_i / T$ 是**无量纲**的比值（本题 S = 2.885），与吞吐率的量纲（条/秒）完全不同。考生若将加速比数值误当作吞吐率，得到 28.85 $\times 10^8$。
- **C 错误**：各段不等时流水线的时钟周期应由**瓶颈段（最大延迟段）**决定，而非取各段平均。误取 avg=3.0 ns 使总时间变短（162 ns），吞吐率虚高。
- **D 错误**：总执行时间 T 中，首条指令需要经过全部 k 段（$\Sigma\Delta t_i$ = 15 ns 的填充时间）。仅用 $N \times \Delta t_{max}$ = 250 ns 漏掉了这 15 ns 的填充（还遗漏了流水线填充阶段的结构），使总时间偏小、吞吐率偏高。

11.

答案：C　**解析**：计数器定时查询方式通过计数器的值来确定响应哪个设备，改变计数器的初值和工作方式即可灵活设置优先级，其连线数和复杂度介于链式查询（连线最少）与独立请求方式（连线最多、响应最快）之间，C 正确。A 错误，链式查询方式对电路故障非常敏感（一个设备故障影响后续所有设备），且响应速度不是最快的；B 错误，独立请求方式连线最多，且优先级可由总线仲裁器灵活设置；D 错误，独立请求方式响应速度最快，因为它不需要逐个查询。

12.

公式：总线带宽 = 总线宽度（Byte）× 总线工作频率

第 1 步：将总线宽度由 bit 换算为 Byte：

$W = 32\ \text{bit} \div 8 = 4\ \text{Byte}$

第 2 步：计算带宽：

$B = 4\ \text{Byte} \times 200\ \text{MHz} = 800\ \text{MB/s}$

> **A. 800 MB/s** ✅（正确答案）

B. 6400 MB/s —— 忘记 ÷8 换算

错误路径：直接用 bit 宽度乘频率，未将 bit 换算为 Byte。

$B_{\text{错}} = 200 \times 32 = 6400$

该值是正确值的 **8 倍**（恰为 W/8 的倒数倍数），是此类题最高频的错误。

C. 800000 MB/s —— 单位混淆（MHz 误当 GHz）

错误路径：将 200 MHz 的数值直接当作 GHz 量级理解，等价于频率被放大 1000 倍。

$B_{\text{错}} = (200 \times 1000) \times 4 = 800000$

该值是正确值的 **1000 倍**，产生明显的数量级偏差。

D. 1 MB/s —— 公式混淆（乘除颠倒）

错误路径：误用 "总线宽度 × 8 ÷ 频率"（乘除颠倒）。

$B_{\text{错}} = 32 \times 8 \div 200 = 1$

该值异常小，因公式中乘除完全颠倒所致。

选项	值	说明
A	**800 MB/s**	**正确答案**：先 ÷8 换算为 Byte，再乘频率
B	6400 MB/s	未 ÷8，正确值的 8 倍
C	800000 MB/s	MHz 误当 GHz，正确值的 1000 倍
D	1 MB/s	乘除颠倒，W×8/F

正确答案：A（800 MB/s）

13.

答案：B　**解析**：CPU　在每条指令执行完毕时检测中断请求信号，当中断允许（中断允许触发器 IF=1）且有中断请求时，才在当前指令结束后进入中断响应周期，B 正确。A 错误，CPU 不能在任意时刻打断指令执行，必须等当前指令执行完毕；C　错误，硬件自动完成的是"关中断、保存断点

（PC 和 PSW）到特殊寄存器或栈、将中断服务程序入口地址送入 PC"，但并非所有体系结构都自动将返回地址压栈，这取决于具体实现；D 错误，保护现场（保存通用寄存器内容）通常由中断服务程序中的软件指令完成，硬件一般只负责保存断点（PC/PSW）。

14.

答案：B **解析**：周期挪用（周期窃取）是指 DMA 控制器在 CPU 不占用总线的存储周期（或机器周期）时，挪用一个总线周期进行数据传送，此时如果 CPU 也需要访存则需暂停一个周期，B 正确。A 错误，DMA 的核心特征就是数据直接在外设与主存之间传送，不经过 CPU 寄存器中转；C 错误，DMA 虽然不经过 CPU，但占用总线进行传送，当与 CPU 争用总线时会影响 CPU 效率；D 错误，周期挪用方式下 CPU 仅在争用总线时暂停一个周期，并非始终空闲。

15.

答案：C **解析**：统一编址（存储器映射编址）将 I/O 端口与存储器安排在同一地址空间中，因此不需要专门的 I/O 指令，可用访存指令访问 I/O 端口，代价是占用部分存储器地址空间，C 正确。A 错误，统一编址不需要专门的 I/O 指令；B 错误，独立编址下 I/O 端口与存储器各有独立的地址空间，需要专门的 I/O 指令（如 IN/OUT 指令）；D 错误，独立编址虽然不占用存储器地址空间，但需要专门的 I/O 指令来访问端口。

二、填空题（10分）

1.

空号	答案
①	E_A = **4**，E_B = **2**
②	ΔE = **2**，方向 = **将 B 右移对齐到 A**
③	对阶后 B 尾数主体 = `00111111111111111111111`，G=**1**，R=**1**，S=**0**
④	求和原始结果（含GRS，28位）= **`10001111111111111111111111110`**（整数部分 10）
⑤	方向 = **右规**，规格化后尾数 = **`1.00011111111111111111111`**（GRS=111）
⑥	规格化后阶码真值 = **5**
⑦	舍入后尾数 = **`1.00100000000000000000000`**，溢出 = **否**
⑧	最终移码 = **132**，溢出 = **无溢出**
⑨	IEEE754 编码 = **0x42100000**
⑩	十进制真值 = **36**

三、简答题（5×6=30分）

1.

子问	答案
(1) 加权平均 CPI	**1.7**
(2) MIPS	**470.59**
(3) 执行时间	**0.425 s**

2.

求解：指令格式分析与寻址方式有效地址推导

> 本文档逐字对照 gen.py exec 输出转写，不重算。

SQ1（2分）字段位数与直接寻址范围

（1）OP、M、D 位数确定

字段	推导	位数
OP（操作码）	该机共 12 条指令，需 ceil($\log_2 12$) = 4 位编码	**4 位**
M（寻址特征）	支持 6 种寻址方式，需 ceil($\log_2 6$) = 3 位编码	**3 位**
D（形式地址）	剩余位数 = 16 − 4 − 3	**9 位**

验证：OP + M + D = 4 + 3 + 9 = **16 位**（= 指令字长）。

（2）直接寻址范围

直接寻址时 EA = D，D 为 9 位无符号整数：

- **十进制**：0 ~ 511
- **十六进制**：000H ~ 1FFH

SQ2（3分）6 种寻址方式有效地址

给定参数值：D = 0C0H，IX = 2000H，BR = 3800H，PC = 1F02H（取指后），内存单元 (00C0H) = 0400H。

寻址方式	EA 表达式	计算过程	EA 值（十六进制）
立即寻址	D 本身即为操作数，无有效地址	—	—
直接寻址	EA = D	0C0H	**00C0H**
间接寻址	EA = (D)，即 D 所指单元内容	(00C0H) = 0400H	**0400H**
变址寻址	EA = (IX) + D	2000H + 0C0H = 20C0H	**20C0H**
基址寻址	EA = (BR) + D	3800H + 0C0H = 38C0H	**38C0H**
相对寻址	EA = (PC) + D	1F02H + 0C0H = 1FC2H	**1FC2H**

关键说明：

- **间接寻址**：需额外访存，以 D 的值（00C0H）作为地址读取内存单元，内容为 0400H，即为有效地址。
- **相对寻址**：PC 为**取指后**的值（1F02H，即指向下一条指令），EA = 1F02H + 0C0H = 1FC2H。

SQ3（1分）变址寻址与基址寻址的本质区别

维度	变址寻址	基址寻址
可控方	由**用户/程序员**控制 IX 值	由**操作系统**控制 BR 值
典型用途	面向**数组遍历、循环**：D 固定为数组首地址偏移，不断改变 IX 遍历元素	面向**多道程序、地址重定位**：OS 通过修改基址实现程序在内存中的浮动
本质	D 不变，通过修改变址寄存器 IX 扩展访问范围	BR 不变（由 OS 设定），D 作为段内偏移
寻址空间	扩大数组的可访问范围	扩大物理寻址空间，实现程序重定位

一句话总结：变址寻址中变址寄存器由用户控制、面向数组/循环等数据结构遍历；基址寻址中基址寄存器由操作系统控制、面向程序重定位和多道程序的地址变换。

场景规则验证（gen.py exec assert 全部通过）

规则	内容	验证结果
R1 字段位宽	4+3+9=16；OP≥ceil(log₂12)=4；M≥ceil(log₂6)=3	√
R2 直接寻址范围	0 ~ 2^9-1 = 0 ~ 511	√
R3 D 合法	0C0H=192 < 2^9=512	√
R4 间接单元可达	D=00C0H 在直接寻址范围内	√
R5 EA 互异	直接=00C0H, 间接=0400H, 变址=20C0H, 基址=38C0H, 相对=1FC2H, 全不同	√
R6 相对寻址 PC	EA = PC(取指后) + D = 1F02H + 0C0H = 1FC2H	√

3.

同步总线带宽计算 —— 参考解答

已知参数

参数	值	说明
f	66 MHz	总线时钟频率（非 2 的幂整数）
w	32 bit	总线数据宽度
C	3	单字事务总时钟周期数（含地址周期）
N	4	突发传输数据字数
进制约定	1\ MB/s = 10^6\ B/s	十进制兆字节

第 1 问（4 分）：单字传输模式下的最大数据传输带宽

计算过程

步骤 1：将总线宽度从 bit 转为 Byte（陷阱 2）

w/8 = \frac{32\ bit}{8} = 4\ Byte

每个数据周期可传输 4 字节（1 个数据字）。

步骤 2：确定单字事务总周期数（陷阱 1）

单字事务共需 C = 3 个时钟周期（含 1 个地址周期 + 2 个数据周期）。地址周期占用时间但不传输数据，因此正确分母应为**总周期数** C = 3，而非仅数据周期数 C - 1 = 2。

步骤 3：计算带宽

$BW_{single} = (w/8) \times f/C = 4 \times 66/3 = 264/3 = 88\ MB/s$

> 说明：f = 66 MHz = 66 × 10^6 Hz，w/8 = 4 Byte，相乘得每 C 个周期传输 264 字节 × 10^6，再除以 C = 3，即 88 × 10^6 B/s = 88 MB/s。

Q1 答案

$\boxed{BW_{single} = 88\ MB/s}$

第 2 问（2 分）：突发传输有效带宽与提升倍数

计算过程

步骤 1：确定突发事务总周期数（陷阱 3）

突发传输中，1 个地址周期后连续传输 N = 4 个数据字，每个数据字占用 1 个数据周期：

突发事务总周期数 = 1\ (地址) + N\ (数据) = 1 + 4 = 5

注意：突发模式下地址周期只占 1 次，总周期数为 1 + N = 5，而非 N × C = 12。

步骤 2：计算突发传输数据量

突发数据量 = N × (w/8) = 4 × 4 = 16\ Byte

步骤 3：计算突发带宽

$BW_{burst} = N \times (w/8) \times f/1 + N = 4 \times 4 \times 66/5 = 1056/5 = 211.2\ MB/s$

> 进制验证（陷阱 4）：采用 10^6 字节/秒，16 × 66 × 10^6 / 5 = 211.2 × 10^6 B/s = 211.2 MB/s。

步骤 4：计算带宽提升倍数

$ratio = \frac{BW_{burst}}{BW_{single}} = 211.2/88 = 2.40$

> 物理含义：突发模式将 1 次地址开销分摊到 4 个数据字上，地址开销占比从单字模式的 1/3 降至突发的 1/5，因此带宽提升至 2.40 倍。

Q2 答案

$\boxed{BW_{burst} = 211.2\ MB/s, \quad 提升倍数 = 2.40}$

场景规则验证（全部通过）

规则	验证内容	结果
R1	$BW_{single} = (w/8) \times f / C = 4 \times 66 / 3 = 88$ MB/s	✅ PASS
R2	$BW_{burst} = N \times (w/8) \times f / (1+N) = 16 \times 66 / 5 = 211.2$ MB/s	✅ PASS
R3	$C = 3 \geqslant 2$ 且 $N = 4 > 1$（周期数守恒）	✅ PASS
R4	ratio = 211.2 / 88 = 2.4 > 1（突发模式带宽高于单字模式）	✅ PASS
R5	f = 66 非二幂 √，w = 32 ∈ {8,16,32,64} √，C = 3 ∈ [2,5] √，N = 4 ∈ [4,8] √	✅ PASS

4.

变形补码加减运算与溢出判断 — 求解

参数

- $x = -0.6875 = -\dfrac{11}{16}$
- $y = -0.5625 = -\dfrac{9}{16}$
- 变形补码格式：$S_1S_0.f_1f_2f_3f_4f_5$（2 位符号位 + 5 位小数位）

第 1 问：用变形补码求 x + y

① 十进制小数转二进制

|x| = 0.6875 转二进制（乘 2 取整法）：

步骤	|x| × 2	整数部分	剩余
1	0.6875 × 2 = 1.375	**1**	0.375
2	0.375 × 2 = 0.75	**0**	0.75
3	0.75 × 2 = 1.5	**1**	0.5
4	0.5 × 2 = 1.0	**1**	0.0

$|x| = 0.1011_2$（4 位精确终止），故 $x = -0.1011_2$。

|y| = 0.5625 转二进制：

步骤	|y| × 2	整数部分	剩余
1	0.5625 × 2 = 1.125	**1**	0.125
2	0.125 × 2 = 0.25	**0**	0.25
3	0.25 × 2 = 0.5	**0**	0.5
4	0.5 × 2 = 1.0	**1**	0.0

$|y| = 0.1001_2$（4 位精确终止），故 $y = -0.1001_2$。

补齐 5 位小数：$x = -0.10110_2$，$y = -0.10010_2$。

② 变形补码

x、y 均为负数，符号位为 `11`，数值部分取反加一。

$[x]_{\text{变形补码}}$：- 数值部分：0.10110 \xrightarrow{取反} 1.01001 \xrightarrow{加一} 1.01010 - $[x]_{\text{变形补码}}$ = 11.01010

$[y]_{\text{变形补码}}$：- 数值部分：0.10010 \xrightarrow{取反} 1.01101 \xrightarrow{加一} 1.01110 - $[y]_{\text{变形补码}}$ = 11.01110

③ 变形补码加法运算

\begin{array}{r} 11.01010 \\ +11.01110 \\ \hline 110.11000 \end{array}

详细过程：

```
  11.01010   ([x]变形补码)
+ 11.01110   ([y]变形补码)
-----------
 110.11000    （最高位1溢出/丢弃，保留7位）
```

- 编码值：106 + 110 = 216，取模 \bmod 128 = 88
- 结果编码 = `1011000`，即 **`10.11000`**

④ 溢出判断

结果双符号位为 S_1S_0 = 10（不同）。

根据溢出判断规则：双符号位不同 → **溢出**。

结论：x + y 运算**溢出**。（数学验证：x + y = -0.6875 + (-0.5625) = -1.25，超出本格式变形补码小数表示范围 $[-1, 1-2^{-5})$ = [-1, \tfrac{31}{32})，确实溢出。）

第 2 问：用变形补码求 x - y

① 求 [-y] 及其变形补码

-y = -(-0.5625) = +0.5625 = +\dfrac{9}{16}

二进制表示：$-y = +0.1001_2 = +0.10010_2$（补齐 5 位）。

-y 为正数，符号位 `00`，数值部分即为原码：

$[-y]_{\text{变形补码}}$ = 00.10010

验证：$[-y]_{\text{变形补码}} + [y]_{\text{变形补码}}$ = 00.10010 + 11.01110 = 100.00000，取模后为 00.00000 = 0 ✓

② 变形补码加法运算（x + [-y]）

\begin{array}{r} 11.01010 \\ +00.10010 \\ \hline 11.11100 \end{array}

详细过程：

```
  11.01010   ([x]变形补码)
+ 00.10010   ([-y]变形补码)
-----------
  11.11100    (结果)
```

· 编码值：106 + 18 = 124，取模 \bmod 128 = 124

· 结果编码 = `1111100`，即 **`11.11100`**

③ 溢出判断

结果双符号位为 S_1S_0 = 11（相同）。

根据溢出判断规则：双符号位相同 → **未溢出**。

④ 结果真值

双符号位 `11` 表示负数，数值部分 `11100` 为补码小数：

· 数值部分 `11100` 取反加一还原原码：取反 → `00011`，加一 → `00100`，即 $0.00100_2 = 0.125$

· 负号，故结果真值 = -0.125，即 -\dfrac{1}{8}。

验证：x - y = -0.6875 - (-0.5625) = -0.6875 + 0.5625 = -0.125 ✓

最终结论汇总

问	运算	变形补码结果	双符号位	溢出判断	真值
第1问	x + y	`10.11000`	`10`（不同）	**溢出**	—
第2问	x - y	`11.11100`	`11`（相同）	**未溢出**	-0.125

5.

先行进位加法器进位表达式文字推导 — 参考解答

题目参数（PARAMS）

参数	值
位宽 n	4 位
输入	$A = A_3A_2A_1A_0$、$B = B_3B_2B_1B_0$，最低位下标 0
进位输入	C_0（保留符号，不赋具体值）
进位生成函数	$G_i = A_i \cdot B_i$（与）

参数	值
进位传递函数	$P_i = A_i \oplus B_i$（异或，本题统一采用）
递推公式	$C_{i+1} = G_i + P_i \cdot C_i$
验证试验	20000 次随机注入 (A_i, B_i, C_0)

第 1 问（2 分）— G_i、P_i 定义及递推公式

- **进位生成函数**：$G_i = A_i \cdot B_i$（当 $A_i = B_i = 1$ 时产生进位）
- **进位传递函数**：$P_i = A_i \oplus B_i$（当 A_i 与 B_i 恰好有一个为 1 时，将低位进位向上传递）
- **第 i 级进位递推公式**：

$$C_{i+1} = G_i + P_i \cdot C_i$$

第 2 问（3 分）— 逐级展开 C_1、C_2、C_3

C_1 的展开

由递推公式（$i = 0$）直接写出：

$$C_1 = G_0 + P_0 \cdot C_0$$

C_2 的展开

由递推公式（$i = 1$）写出：

$$C_2 = G_1 + P_1 \cdot C_1$$

将 C_1 的展开式 $C_1 = G_0 + P_0 \cdot C_0$ 代入：

$$C_2 = G_1 + P_1 \cdot (G_0 + P_0 \cdot C_0)$$

展开并合并（分配律），仅保留 G_j、P_j、C_0：

$$\boxed{C_2 = G_1 + G_0 \cdot P_1 + P_0 \cdot P_1 \cdot C_0}$$

代入步骤：$C_2 = G_1 + P_1\,C_1 = G_1 + P_1(G_0 + P_0\,C_0) = G_1 + G_0\,P_1 + P_0\,P_1\,C_0$。

C_3 的展开

由递推公式（$i = 2$）写出：

$$C_3 = G_2 + P_2 \cdot C_2$$

将 C_2 的展开式 $C_2 = G_1 + G_0\,P_1 + P_0\,P_1\,C_0$ 代入：

$$C_3 = G_2 + P_2 \cdot (G_1 + G_0 \cdot P_1 + P_0 \cdot P_1 \cdot C_0)$$

展开并合并（分配律），仅保留 G_j、P_j、C_0：

$$\boxed{C_3 = G_2 + G_1 \cdot P_2 + G_0 \cdot P_1 \cdot P_2 + P_0 \cdot P_1 \cdot P_2 \cdot C_0}$$

代入步骤：$C_3 = G_2 + P_2\,C_2 = G_2 + P_2(G_1 + G_0\,P_1 + P_0\,P_1\,C_0) = G_2 + G_1\,P_2 + G_0\,P_1\,P_2 + P_0\,P_1\,P_2\,C_0$。

（补充）C_4 的展开

> gen.py 完整展开到 n = 4 位，此处列出供完整性参考。

$$C_4 = G_3 + G_2 \cdot P_3 + G_1 \cdot P_2 \cdot P_3 + G_0 \cdot P_1 \cdot P_2 \cdot P_3 + P_0 \cdot P_1 \cdot P_2 \cdot P_3 \cdot C_0$$

第 3 问（1 分）— 先行进位 vs 行波进位的延迟优势

行波进位（串行进位）：第 i 级的进位 C_{i+1} 依赖于第 i-1 级产生的进位 C_i，即每一级的进位都必须等待前一级进位算出后才能开始计算。n 位加法器的进位延迟随位数线性增长（O(n) 级门延迟）。

先行进位（Carry-Lookahead）：通过将各级进位展开为仅含 G_j、P_j 和 C_0 的表达式，所有进位信号都**直接由 C_0 和本级的 G、P 函数产生**，不再依赖中间进位 $C_1, C_2, \cdots$ 的逐级传递。因此所有位的进位信号可以**并行同时产生**，进位延迟降为常数级（O(1)　　级门延迟，仅需少数几级门的固定延迟），大幅缩短了进位链路。

场景规则验证（RULE_CHECK）

gen.py 对四条场景规则进行了验证（20000 次随机布尔注入 A_i, B_i, C_0）：

规则	内容	结果
规则1 递推一致性	展开后 C_k 的布尔值 == $G_{k-1} + P_{k-1} \cdot C_{k-1}$ 的递推值（k=1,2,3,4）	√ PASS
规则2 展开完整性	C_k 展开式中仅含 G_0…G_{k-1}、P_0…P_{k-1}、C_0，无中间进位残留	√ PASS
规则3 P_i 定义一致性	全题统一 $P_i = A_i \oplus B_i$，数值验证 XOR 版本正确（对比 $A_i + B_i$ 版本会不一致）	√ PASS
规则4 下标单调性	C_k 展开式中 G、P 下标 $\in [0, k-1]$，无越界（如 C_3 不含 G_3、P_3）	√ PASS

最终先行进位表达式（RESULT）

$$C_1 = G_0 + P_0 \cdot C_0$$

$C_2 = G_1 + G_0 \cdot P_1 + P_0 \cdot P_1 \cdot C_0$

$C_3 = G_2 + G_1 \cdot P_2 + G_0 \cdot P_1 \cdot P_2 + P_0 \cdot P_1 \cdot P_2 \cdot C_0$

$C_4 = G_3 + G_2 \cdot P_3 + G_1 \cdot P_2 \cdot P_3 + G_0 \cdot P_1 \cdot P_2 \cdot P_3 + P_0 \cdot P_1 \cdot P_2 \cdot P_3 \cdot C_0$

四、应用题（10分）

1.

已知参数

参数	值
主存容量 M	1 MB = 2^{20} B
Cache 容量 C	32 KB = 2^{15} B
主存块大小 B（Cache 行大小）	64 B = 2^6 B
组相联路数 W	4 路

按字节编址，主存地址总位数 = $\log_2(M) = \log_2(2^{20})$ = **20 位**。

第 1 问（3 分）— 直接相联映射

1. 块内偏移字段位数

块内偏移 = $\log_2(B) = \log_2(2^6)$ = **6 位**。

计算依据：每个 Cache 行大小为 64 B，按字节编址时需要 6 位地址来寻址行内的每一个字节（2^6 = 64）。

2. Cache 行号字段位数

· Cache 总行数 = C / B = 32768 / 64 = 512 行

· 行号位数 = $\log_2(C/B) = \log_2(512)$ = **9 位**

计算依据：直接映射下每个主存块只能映射到 Cache 中固定的一行，需要 9 位从 512 行中选一行（2^9 = 512）。

3. Tag 字段位数

Tag 位数 = 地址总位数 − 行号位数 − 块内偏移位数 = 20 − 9 − 6 = **5 位**。

计算依据：地址总位数 20 位减去行号字段 9 位和块内偏移字段 6 位，剩余 5 位即为标记字段。

字段守恒验证：Tag(5) + 行号(9) + 块内偏移(6) = 20 = 地址总位数 √

第 2 问（4 分）— 4 路组相联映射

1. 块内偏移字段位数

块内偏移 = $\log_2(B)$ = **6 位**。

与第 1 问相同。理由：块内偏移仅由块大小 B 决定（$2^6 = 64$），与映射方式无关，两种映射方式的块内偏移必然相同。

2. 组号字段位数

- Cache 总行数 = C / B = 32768 / 64 = 512 行
- 组数 = C / (B × W) = 32768 / (64 × 4) = 32768 / 256 = 128 组
- 组号位数 = $\log_2(128)$ = **7 位**

计算依据：4 路组相联将 512 行按每组 4 行分组，共 128 组（512 / 4 = 128），需要 7 位从 128 组中选一组（$2^7 = 128$）。

3. Tag 字段位数

Tag 位数 = 地址总位数 − 组号位数 − 块内偏移位数 = 20 − 7 − 6 = **7 位**。

计算依据：地址总位数 20 位减去组号字段 7 位和块内偏移字段 6 位，剩余 7 位即为标记字段。

字段守恒验证：Tag(7) + 组号(7) + 块内偏移(6) = 20 = 地址总位数 √

第 3 问（3 分）— Tag 位数差异比较

1. 两种映射方式的 Tag 位数差

- 直接相联 Tag = 5 位
- 组相联 Tag = 7 位
- 差值 = 7 − 5 = **2 位**（组相联的 Tag 比直接相联多 2 位）

2. 产生差异的原因

组相联的索引字段（组号 7 位）比直接相联的索引字段（行号 9 位）少了 2 位（因为 4 路组相联将行数除以路数 4，组数 = 512/4 = 128 < 512 行），索引字段减少的位数转移到 Tag 字段上，因此组相联的 Tag 位数比直接相联多 2 位。

字段	第1问 直接相联	第2问 4路组相联
块内偏移	6 位	6 位
索引字段（行号/组号）	行号 9 位	组号 7 位
Tag	**5 位**	**7 位**
合计	20 位	20 位

第 3 问：组相联 Tag 比直接相联 Tag 多 **2 位**，原因是组相联索引字段（组号）比直接相联索引字段（行号）少 2 位（路数 4 = 2^2），减少的位数转移到 Tag 上。

五、设计题（2×10=20分）

1.

项目	结果
各字段位数	F_1=3, F_2=3, F_3=2, F_4=3, F_5=2
操作控制字段总位数	13 位
下址字段位数	8 位
判别测试位数	2 位
微指令字长	**23 位**
控存最大容量	**5888 位**

2.

DRAM 芯片存储器扩展设计 —— 求解

已知参数

参数	值
CPU 地址总线宽度	20 位
CPU 数据总线宽度	16 位
编址方式	按字编址（每地址 = 16 位存储单元）
DRAM 芯片规格	64K×4 位
刷新间隔	2 ms
刷新方式	异步刷新

问题 1：芯片数计算（3 分）

① 位扩展数（每组芯片数）：

CPU 数据总线 16 位，芯片位宽 4 位，需要位扩展使数据位匹配：

位扩展数 = \frac{数据总线宽度}{芯片位宽} = 16/4 = 4 片/组

② 片内地址位数：

片内地址位数 = $\log_2(64K) = \log_2(2^{16}) = 16$ 位

③ 片选地址线数：

片选地址线数 = 20 - 16 = 4 位

④ 字扩展数（芯片组数）：

字扩展数 = $2^{\text{片选线数}} = 2^4 = 16$ 组

⑤ 总芯片数：

总芯片数 = 位扩展数 × 字扩展数 = 4 × 16 = \boxed{64 片}

计算依据： - 位扩展：芯片位宽 4 位 < 数据总线 16 位，需 16÷4=4 片并联，使每组输出 16 位数据。 - 字扩展：芯片字数 64K（地址 16 位），CPU 地址空间 2^20 字，需 2^(20-16)=16 组串联，覆盖全部地址空间。

问题 2：地址线划分（3 分）

地址线分配：

用途	地址线	位数
片内地址	A0～A15	16 位
— 行地址（低半）	A0～A7	8 位
— 列地址（高半）	A8～A15	8 位
片选地址	A16～A19	4 位

说明： - 芯片地址复用：片内地址 16 位分两半，行地址 8 位（A0～A7）、列地址 8 位（A8～A15），分两次输入（先送行地址、再送列地址）。 - 片选地址 4 位（A16～A19）用于区分 16 个芯片组。

各芯片组地址范围（十六进制）：

每组大小 = 2^16 = 64K 个地址，A16～A19 的取值决定芯片组编号。

芯片组	A19A18A17A16	地址范围
第 0 组	0000	0x00000 ～ 0x0FFFF
第 1 组	0001	0x10000 ～ 0x1FFFF
第 2 组	0010	0x20000 ～ 0x2FFFF
第 3 组	0011	0x30000 ～ 0x3FFFF
第 4 组	0100	0x40000 ～ 0x4FFFF
第 5 组	0101	0x50000 ～ 0x5FFFF

芯片组	A19A18A17A16	地址范围
第 6 组	0110	0x60000 ～ 0x6FFFF
第 7 组	0111	0x70000 ～ 0x7FFFF
第 8 组	1000	0x80000 ～ 0x8FFFF
第 9 组	1001	0x90000 ～ 0x9FFFF
第 10 组	1010	0xA0000 ～ 0xAFFFF
第 11 组	1011	0xB0000 ～ 0xBFFFF
第 12 组	1100	0xC0000 ～ 0xCFFFF
第 13 组	1101	0xD0000 ～ 0xDFFFF
第 14 组	1110	0xE0000 ～ 0xEFFFF
第 15 组	1111	0xF0000 ～ 0xFFFFF

全部 16 组地址范围连续覆盖 CPU 地址空间 [0x00000, 0xFFFFF]（即 0 ～ 2^20−1），无重叠、无遗漏。

问题 3：片选逻辑（2 分）

片选地址线为 A16～A19（共 4 根），需区分 16 个芯片组。

设计方案：

使用一片 **4:16 译码器**（4 线-16 线译码器），其 4 个输入端分别接 A16、A17、A18、A19，16 个输出端 Y0～Y15 分别连接第 0～15 组芯片的片选信号 CS。

对应关系：

译码器输入 A19A18A17A16	译码器有效输出	选中的芯片组
0000	Y0	第 0 组
0001	Y1	第 1 组
…	…	…
1111	Y15	第 15 组

当 CPU 给出某地址时，A16～A19 选中译码器的某一输出，该输出为低电平（有效），使能对应的芯片组；其余 15 组未被选中。

> 说明：若无现成 4:16 译码器芯片，也可用 2 片 3:8 译码器级联（用 A19 作为使能控制，区分高半和低半各 8 组），逻辑等效。

问题 4：异步刷新周期（2 分）

① 芯片行数：

行地址位数 = 片内地址位数 ÷ 2 = 16 ÷ 2 = 8 位（地址复用，行、列各 8 位）。

芯片行数 = $2^{\text{行地址位数}} = 2^8 = 256$ 行

② 异步刷新周期：

异步刷新在 2 ms 内将所有 256 行均匀刷新一遍，两次刷新之间的时间间隔为：

刷新周期 = \frac{刷新间隔}{行数} = \frac{2 ms}{256} = \frac{2000 μs}{256} = \boxed{7.81 μs}

（精确值 7.8125 μs，约 7.81 μs）

场景规则验证汇总

规则	验证结果
① 位扩展 = 16/4 = 4	√ 整数且 ⩾2
② 片内地址 = log2(65536) = 16	√
③ 片选线 = 20−16 = 4	√ ⩾2
④ 字扩展 = 2^4 = 16	√ ⩾2
⑤ 总芯片 = 4×16 = 64	√
⑥ 地址复用 = 行8 列8	√ 整数
⑦ 行数 = 2^8 = 256	√
⑧ 刷新周期 = 2000μs/256 = 7.81μs	√
⑨ 地址覆盖 [0, 0xFFFFF]	√ full_cover=True